

TECHNICAL DESIGN & IMPLEMENTATION RUNBOOK

Private AI Assistant for Frappe / ERPNext

A self-hosted, isolated assistant over selected ERP data —
hybrid text-to-SQL + RAG, document OCR, image Q&A, and pest
identification.

Prepared for: Upande · james@upande.com

Phase focus: Free evaluation on **Lightning AI** — no payment until a model is proven

Version: 1.0 | **Date:** 13 July 2026 | **Status:** Approved design → runbook

Contents

1. Executive Summary

Goals, the free-first principle, and what we deliver

2. System Architecture

Full system diagram · Query data flow · Component responsibilities

3. Project Decomposition & Sequencing

Specs A–D · Dependency graph · Build order

4. Spec A — VLM Brain Comparison Harness

Candidates · Sample data · Eval set · Scoring

5. Spec B — Pest Identification CV Model

Transfer learning · IP102 · Training & inference

6. The Free Test Plan & Lightning AI

Three stages · CPU/GPU split · Studio workflow

7. Runbook — Commands & Resources

Lightning setup · Serving · Retrieval · OCR · Eval · Pest training

8. Cost Model & Migration to Paid

9. Risks, Constraints & Decisions

10. Appendix — Resource Index

1 • Executive Summary

Build a privately-hosted AI assistant that answers questions over a **curated, isolated mirror** of selected Frappe/ERPNext tables — using structured data, free text, documents, and images — without exposing the production ERP and without per-token API bills.

1.1 Goals

#	Goal	How this design meets it
G1	Answer questions from ERP structured data	Text-to-SQL over a mirror DB (execution-verified)
G2	Answer from free text / notes	Vector RAG over embedded text fields
G3	Read documents (invoices, delivery notes)	OCR via VLM or dedicated engine → structured fields
G4	Image Q&A (describe / understand photos)	Vision-language model (same brain)
G5	Identify pests from photos	Dedicated fine-tuned CV classifier (Track B)
G6	Data isolation — model never touches production	Only selected tables sync to a separate box the model sees
G7	No cost until proven	All evaluation on free tiers; pay only to deploy the winner

1.2 The free-first principle

Decouple evaluation from production. Choosing a model is a one-off experiment; it does *not* need a paid, isolated box. We screen and validate every candidate on **free infrastructure** using **synthetic data**, and spend the first real money only when we already know which model to deploy. This document scopes the free phase on **Lightning AI**.

1.3 What "self-hosted only" commits us to

The isolation requirement (G6) rules out hosted APIs as the *production* brain, because sending even the synced subset to a third party breaks isolation. Production therefore needs a GPU box we control (rented ~16 GB to start). But — see §6 — the **comparison** is still free because it runs on synthetic data on free compute.

1.4 Deliverables of the free phase

- A **scorecard** ranking VLM candidates on SQL accuracy, RAG quality, OCR, image-Q&A, latency and VRAM fit → one recommended brain + runner-up.
- A **pest classifier** with a per-pest accuracy report, trained on public data.
- A reusable **harness** and **Lightning Studio** environment, ready to re-run when new models appear.

2 · System Architecture

The system is one **isolated environment** that the model controls, fed by a one-way sync from Frappe. The model can only ever see what was synced — its blast radius is the curated subset, never the live ERP.

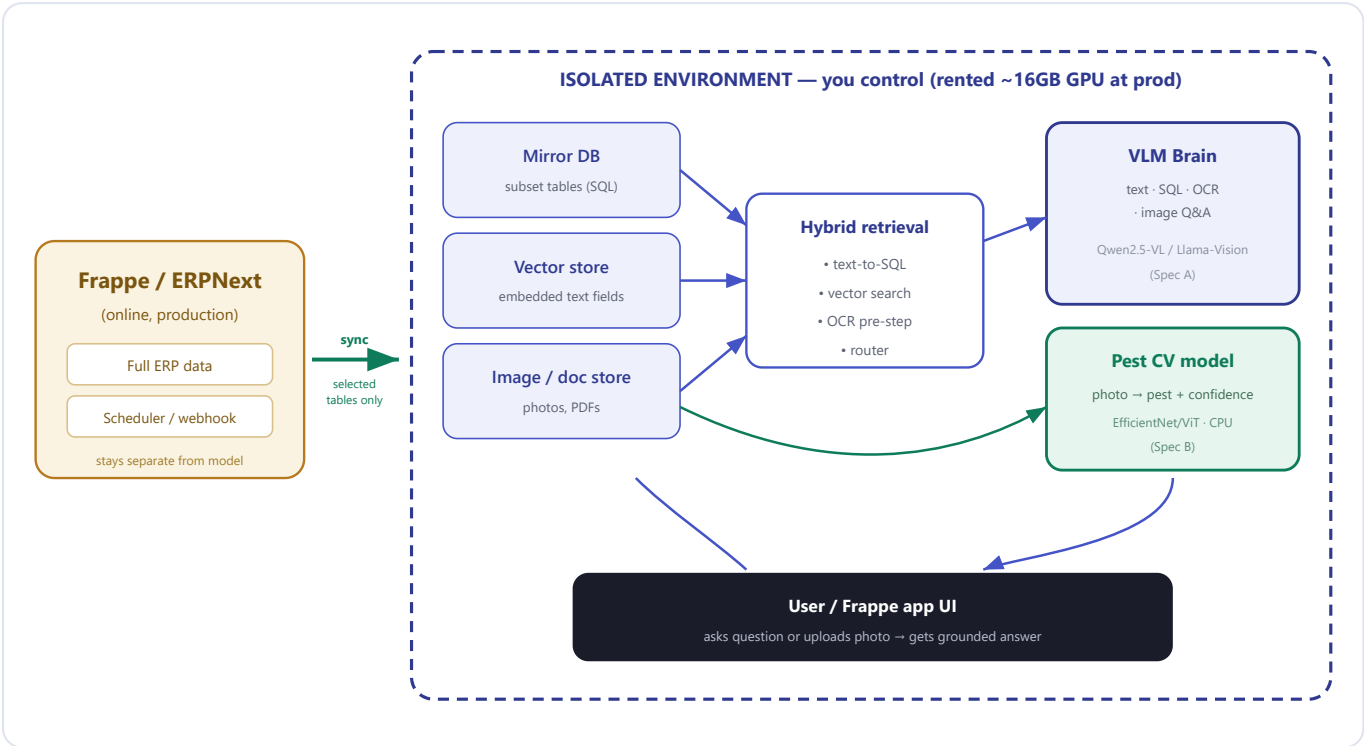


Figure 1 — Full system. One-way sync (green) copies only selected tables into an isolated box; the model never reaches production Frappe.

2.1 Query data flow (hybrid)

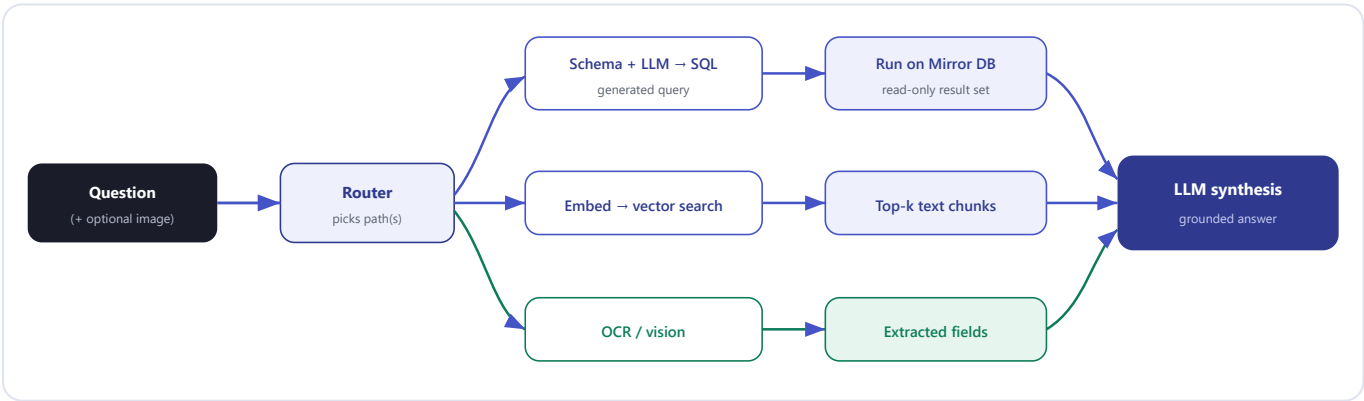


Figure 2 — A question is routed to one or more paths; results are fused and the LLM writes a grounded answer.

2.2 Component responsibilities

Component	Responsibility	Depends on
Sync job	Push selected Frappe tables/records into the mirror on a schedule/webhook	Frappe API
Mirror DB	Read-only structured store the SQL runs against	Postgres/SQLite

Vector store	Embeddings of text fields for semantic search	Embedding model
Hybrid retriever	Route question, run SQL + vector search + OCR, assemble context	Mirror DB, vector store, VLM
VLM brain	Generate SQL, read images/docs, synthesize answers	GPU (prod) / free tier (eval)
Pest CV model	Classify pest from a photo	CPU only

3 · Project Decomposition & Sequencing

The system is split into four sub-projects, each its own spec → plan → build cycle. Only **A** and **B** are in the free phase; **C** and **D** follow once a brain is chosen.

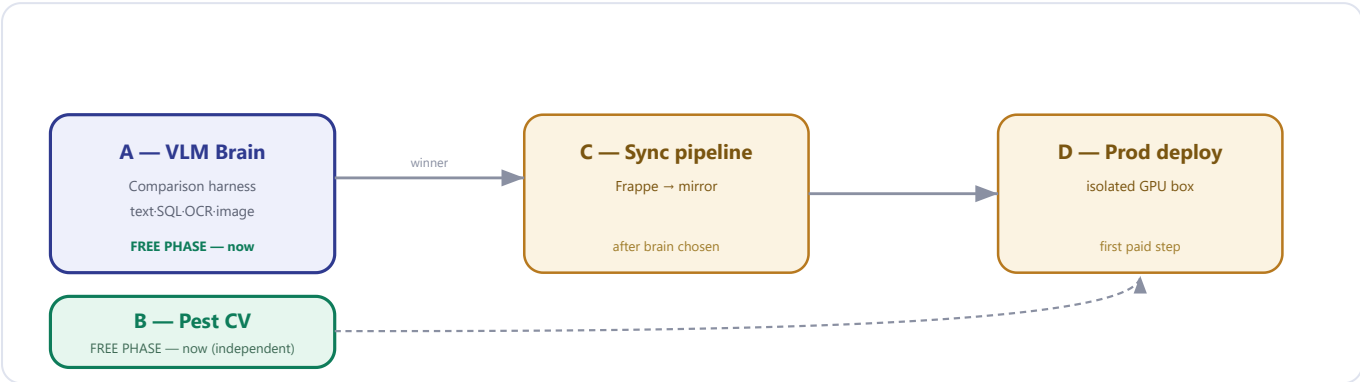


Figure 3 — Dependency graph. A and B run free and in parallel; C and D begin only after the winning brain is known (D = first paid step).

Spec	Scope	Compute	Cost	Order
A	Pick best VLM brain	Free APIs → Lightning T4	\$0	1 (with B)
B	Pest classifier	Free Kaggle/Lightning T4; CPU inference	\$0	1 (with A)
C	Frappe → mirror sync	CPU	\$0	2
D	Isolated prod + serve	Rented ~16GB GPU	PAID	3

4 · Spec A — VLM Brain Comparison Harness

Goal: objectively pick the best ~16 GB self-hosted **vision-language model** that handles text RAG + text-to-SQL + OCR + image Q&A in one model — evaluated at **\$0** on synthetic data.

4.1 Candidate models

Model	Params	Strengths	~16GB fit (quantized)
Qwen2.5-VL-7B-Instruct	7B	Strong text, SQL, OCR, image — top unified pick	Yes (AWQ/GPTQ)
Llama-3.2-11B-Vision-Instruct	11B	Solid VLM all-rounder	Tight (Q4)
InternVL2-8B	8B	Excellent OCR / doc understanding	Yes
MiniCPM-V 2.6	8B	Strong OCR, efficient	Yes
Qwen2.5-Coder-7B (reference)	7B	Text/SQL only — measures what vision "costs" in SQL skill	Yes

VRAM note: VLMs spend memory on image tokens, so ~16 GB is tight. The harness records real VRAM per model; if the best model needs >16 GB we surface what a ~24 GB tier would buy before you commit.

4.2 Sample data (synthetic — no real ERP data)

- ERPNext-shaped tables: `Customer`, `Sales Invoice`, `Item`, `Delivery Note` — structured fields + free-text (notes/remarks).
- A handful of sample **documents** (invoice/delivery-note PDFs or images) for OCR.
- A few sample **images** for image-Q&A.

4.3 Eval set (~40 questions, with gold answers)

Bucket	Example	Gold form
SQL	"Total sales by customer last quarter"	Correct result set
RAG	"What did the notes say about customer X's complaint?"	Reference answer
Hybrid	"Which overdue customers also had quality complaints?"	Set + reference
OCR	"Extract invoice no., date, total from this PDF"	Field values
Image Q&A	"Describe what's in this photo"	Reference answer

4.4 Scoring

Dimension	Metric	Method
SQL correctness	Execution accuracy	Run generated SQL on mirror; compare result set to gold

SQL safety	Read-only rate	Reject/flag any DDL/DML; must be SELECT-only
RAG / image	Judge score + hit-rate	LLM-as-judge (free API) on correctness & groundedness; retrieval hit-rate
OCR	Field accuracy	Extracted field vs gold (exact/fuzzy)
Operational	tok/s, VRAM, fits-16GB	Measured during Stage 2 runs

5 · Spec B — Pest Identification CV Model

Goal: a photo → pest name + confidence, for common flower/rose pests (thrips, aphids, whitefly, spider mites, caterpillars...). Built by **transfer learning** — no training from scratch.

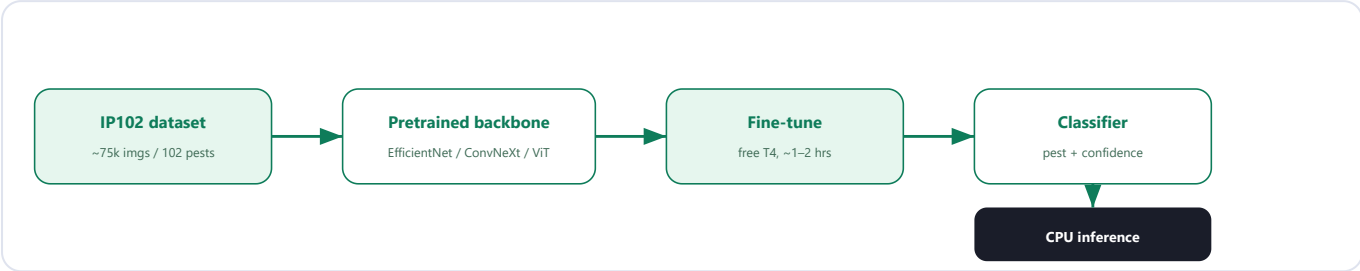


Figure 4 — Pest track. Fine-tune a pretrained backbone on public IP102 data; the resulting classifier is small enough to run on CPU.

Aspect	Choice
Task	Image classification (extendable to detection later)
Data	IP102 (public); PlantVillage / PlantDoc for disease later
Model	Transfer-learn EfficientNet-B0/ConvNeXt/ViT, or Ultralytics YOLO-cls
Train compute	FREE Kaggle/Colab/Lightning T4, ~1–2 hrs
Inference	CPU — runs on the isolated box or even an 8GB laptop
Metrics	Top-1 / Top-3 accuracy, per-class F1, confusion matrix

6 · The Free Test Plan & Lightning AI

Three stages. The first spend of money is Stage 3, by which point the winning model is already proven.

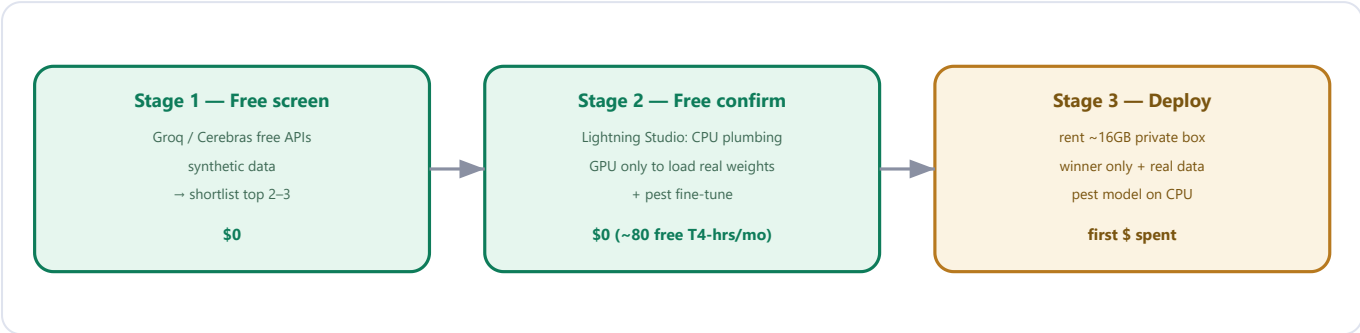


Figure 5 — Free-first staging. Only the proven winner reaches the paid box.

6.1 Why Lightning AI for the free phase

A Lightning "Studio" is a persistent Linux dev box in the browser (VSCode/Jupyter/terminal). Unlike Colab/Kaggle notebooks it **keeps your files and installs between sessions**, and you toggle the GPU on only when needed.

Free-tier fact	Value
Monthly credits	15, recurring (~80 T4-hours on cheap GPU)
Persistence	Files/installs survive between sessions
CPU time	Effectively free — do all setup here, burn no GPU credits
Limits	1 active Studio, interruptible, ~4-hr restart on free

6.2 The credit-saving split

Rule: build every non-model piece (mirror DB, vector store, retrieval, harness, sync stubs) on **free CPU**, using a **free hosted-API brain** (Groq/Cerebras) as a stand-in. Switch the Studio to a **T4 GPU only** to (a) load the shortlisted quantized VLMs for the final scorecard and (b) fine-tune the pest model. Your ~80 free T4-hrs go entirely to the two things that truly need a GPU.

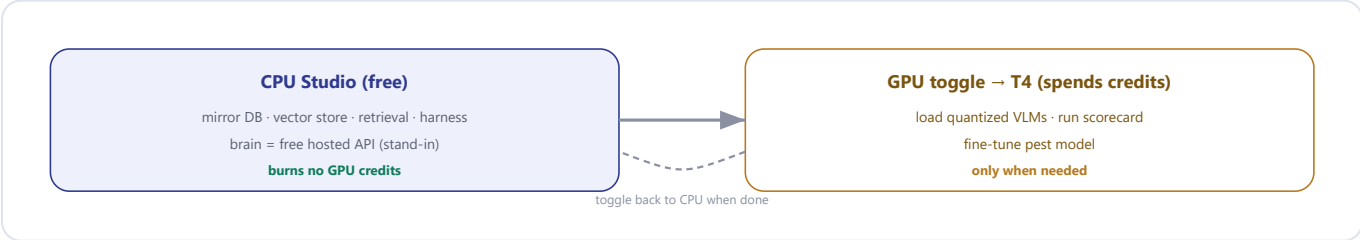


Figure 6 — Same Studio, two machine types. Live on CPU; visit the GPU only for model work.

7 · Runbook — Commands & Resources

Everything below runs **inside the Lightning Studio terminal** unless noted. Commands are grouped by stage.

7.1 Lightning AI — create the environment

1. Sign up at lightning.ai → **New Studio** (starts on CPU by default).
2. Open the Studio → you get VSCode + a terminal in the browser.
3. Switch machine type anytime via the compute selector (top-right): **CPU** (free) ↔ **T4 GPU** (credits).

```
# One-time project scaffold (run on the free CPU machine)
mkdir -p ~/frappe-ai/{data,models,harness,vectorstore,pest}
cd ~/frappe-ai
python -m venv .venv && source .venv/bin/activate
pip install --upgrade pip wheel

# Optional: Lightning CLI for scripted control from your laptop
pip install lightning-sdk
```

7.2 Stage 1 — Free API screening (CPU)

```
pip install openai groq cerebras-cloud-sdk

# Groq (hosts Llama / Qwen) — get key at console.groq.com
export GROQ_API_KEY="gsk..."

# Cerebras (Llama, Qwen3; ~1M tokens/day free) — cloud.cerebras.ai
export CEREBRAS_API_KEY="csk..."
```

```
# harness/screen.py — same prompt across candidates (OpenAI-compatible)
from openai import OpenAI
groq = OpenAI(base_url="https://api.groq.com/openai/v1", api_key=os.environ["GROQ_API_KEY"])
r = groq.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": sql_prompt(schema, question)}])
print(r.choices[0].message.content)
```

Privacy: Stage 1 sends the **synthetic** sample to Groq/Cerebras — never real ERP rows. It is a screen only; the verdict comes from Stage 2 self-hosted runs.

7.3 Stage 2 — Serve real weights (GPU toggle)

Option A — vLLM (best for Qwen2.5-VL; OpenAI-compatible)

```
pip install vllm

# Switch Studio to T4 first. Quantized VLM to fit ~16GB:
vllm serve Qwen/Qwen2.5-VL-7B-Instruct-AWQ \
    --max-model-len 8192 --gpu-memory-utilization 0.92 --port 8000

# endpoint: http://localhost:8000/v1 (same client code as Stage 1)
```

Option B — Ollama (simplest; good for quick multi-model swaps)

```
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull llama3.2-vision:11b
ollama pull minicpm-v
ollama pull qwen2.5vl:7b          # if available in your Ollama version
# chat/vision endpoint at http://localhost:11434
```

7.4 Retrieval layer (CPU — build once)

```
pip install sentence-transformers chromadb sqlalchemy pandas

# Embeddings + local vector store
from sentence_transformers import SentenceTransformer
import chromadb
emb = SentenceTransformer("BAAI/bge-small-en-v1.5")      # small, CPU-fine
client = chromadb.PersistentClient(path=~/.frappe-ai/vectorstore")
col = client.get_or_create_collection("notes")
col.add(ids=ids, embeddings=emb.encode(texts).tolist(), documents=texts)

# Mirror DB for testing (SQLite; swap for Postgres at prod)
from sqlalchemy import create_engine, text
engine = create_engine("sqlite:///~/.frappe-ai/data/mirror.db")
with engine.connect() as c:
    rows = c.execute(text(model_generated_sql)).fetchall()  # execution-accuracy check
```

7.5 OCR (CPU — dedicated engine baseline)

```
pip install paddleocr paddlepaddle          # or: pip install pytesseract (+ system tesseract)
from paddleocr import PaddleOCR
ocr = PaddleOCR(use_angle_cls=True, lang="en")
result = ocr.ocr("data/invoice_01.png")      # compare vs VLM-OCR in the harness
```

7.6 Scoring (LLM-as-judge, free API)

```
# Grade RAG/image answers against reference with a free judge model
judge = groq.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": judge_prompt(question, reference, candidate_answer)}])
# parse {correctness:0-5, grounded:bool} → aggregate into scorecard.csv
```

7.7 Spec B — Pest model training (GPU toggle, ~1–2 hrs)

```
# IP102: available on Kaggle ('ip102') and Hugging Face. Download into pest/data.
pip install ultralytics timm torch torchvision scikit-learn

# Fast path — Ultralytics YOLO classification
yolo classify train model=yolo11n-cls.pt data=~/.frappe-ai/pest/data epochs=30 imgsz=224
yolo classify val   model=runs/classify/train/weights/best.pt data=~/.frappe-ai/pest/data
```

```
# Inference (CPU – runs anywhere, incl. an 8GB laptop)
yolo classify predict model=best.pt source=leaf_photo.jpg
```

```
# Alt path – timm transfer learning (EfficientNet/ConvNeXt), more control
import timm, torch
model = timm.create_model("efficientnet_b0", pretrained=True, num_classes=N_PESTS)
# standard train loop; export best; evaluate top-1/top-3 + confusion matrix
```

8 · Cost Model & Migration to Paid

Phase	Where	What runs	Cost
Stage 1 screen	Groq / Cerebras free	Candidate shortlisting	\$0
Stage 2 confirm	Lightning free (CPU + ~80 T4-hrs)	Full harness, real weights, pest fine-tune	\$0
Stage 3 deploy	RunPod / Vast ~16GB (T4/A10)	Winner brain, real synced data	~\$0.20–0.40/HR
Pest inference	CPU on same box	Classifier serving	NEGLIGIBLE

8.1 When to leave Lightning for a paid box

- The scorecard names a winner and it **fits your VRAM target**.
- You need **always-on** serving (Lightning free has a 4-hr restart and is interruptible).
- You're ready to connect **real** Frappe data (Spec C) — which must never touch a free/shared tier.

Migration is a lift-and-shift: the Studio already has your serving command (vLLM/Ollama), retrieval code, and harness. On the paid box you re-run the same install script, point the mirror DB at real synced data, and serve the single winning model.

9 · Risks, Constraints & Decisions

Risk / constraint	Impact	Mitigation
VLM > 16GB VRAM	Winner won't fit cheap box	Harness reports VRAM; keep ~24GB tier as reference; quantize (AWQ/GPTQ)
Lightning free interruptible + 4-hr restart	Long runs interrupted	Checkpoint harness progress; batch eval in <4-hr chunks
Free API rate limits (Stage 1)	Slow screening	Screen is coarse; throttle + cache responses
Text-to-SQL hallucinated/unsafe SQL	Wrong answers / writes	SELECT-only allow-list; execution-verify; read-only DB user
Pest classes absent from IP102	Can't identify some pests	Confirmed scope = common pests (in IP102); collect own images later if gaps
Isolation vs hosted quality	Self-host weaker than frontier APIs	Accepted: isolation is the hard requirement (G6)

9.1 Key decisions locked

- **Self-hosted open weights only** for production (isolation over convenience).
- **Hybrid** retrieval (SQL + vector), not one or the other.
- **One VLM brain** unifies text + SQL + OCR + image Q&A; pest ID is a **separate** CV model.
- **~16GB** production target; free eval on the equivalent T4.
- **Free-first**: no payment until a model is proven on Lightning.

9.2 Open items to confirm

- Synthetic sample data vs a sanitized dump of your real table shapes.
- Whether we draft the ~40 eval questions or you supply real user questions.
- Keep a ~24GB/32B tier in the comparison as a "what more money buys" reference?

10 · Appendix — Resource Index

Resource	Use	Link
Lightning AI	Free persistent Studio (CPU + ~80 T4-hrs/mo)	lightning.ai
Groq	Free API screening (Llama/Qwen)	console.groq.com
Cerebras	Free API (~1M tokens/day)	cloud.cerebras.ai
Kaggle	Backup free GPU (~30 hrs/wk) + IP102 dataset	kaggle.com
vLLM	Serve VLMs, OpenAI-compatible	docs.vllm.ai
Ollama	Simple local model serving	ollama.com
Qwen2.5-VL	Top unified brain candidate	huggingface.co/Qwen
sentence-transformers / bge-small	Embeddings for RAG	huggingface.co/BAAI
ChromaDB	Local vector store	trychroma.com
PaddleOCR	Dedicated OCR baseline	github.com/PaddlePaddle/PaddleOCR
IP102	Pest classification dataset	Kaggle / HF search "IP102"
Ultralytics	YOLO classification training	docs.ultralytics.com
RunPod / Vast.ai	Paid isolated GPU (Stage 3)	runpod.io · vast.ai

Next step: on approval, this design becomes an implementation plan for **Spec A** (the brain comparison harness) — the first buildable, entirely-free piece — followed by **Spec B** (pest model).